

DIGITALIZAÇÃO E ANÁLISE DE ALGORITMOS

Transcrição dos pseudocódigos manuscritos, avaliação lógica e implementação em Python

Exercício 1: Conversor de Tempo

Avaliação Crítica: O algoritmo está correto e utiliza corretamente os operadores de divisão inteira e resto para fracionar o tempo total em dias, horas, minutos e segundos. O teste de mesa para $t = 90125$ resulta exatamente em 1 dia, 1:02:05, validando a lógica.

PSEUDOCÓDIGO TRANSCRITO

- **Variáveis:** t , $dias$, $horas$, $minutos$, $segundos$, $resto1$, $resto2$ (Inteiros)
- **Entrada:** Ler (t)
- **Processamento:**
 $dias \leftarrow t // 86400$
 $resto1 \leftarrow t \% 86400$
 $horas \leftarrow resto1 // 3600$
 $resto2 \leftarrow resto1 \% 3600$
 $minutos \leftarrow resto2 // 60$
 $segundos \leftarrow resto2 \% 60$
- **Saída:** Escrever ($dias$, "dias,", $horas$, ":", $minutos$, ":", $segundos$)

CÓDIGO PYTHON

```
# Entrada
t = int(input("Digite o tempo em segundos: "))

# Processamento
dias = t // 86400
resto1 = t % 86400

horas = resto1 // 3600
resto2 = resto1 % 3600

minutos = resto2 // 60
segundos = resto2 % 60

# Saída
print(f"{dias} dias, {horas}:{minutos:02d}:{segundos:02d}")
```

Exercício 2: Ponte de Wheatstone

Avaliação Crítica: A modelagem física para o cálculo das resistências equivalentes em paralelo e em série está correta. A condição de equilíbrio ($R1 \times Rx = R2 \times R3$) foi bem aplicada. Incluiu-se a verificação preventiva para impedir divisões por zero caso $R1 = 0$.

PSEUDOCÓDIGO TRANSCRITO

- **Variáveis:** $R1, R2, R3, V_{\text{fonte}}, Rx, R_{\text{total}}, I_{\text{total}}, P_{\text{total}}$ (Reais)
- **Entrada:** Ler ($R1, R2, R3, V_{\text{fonte}}$)
- **Processamento:**
 Se $R1 = 0$ Então Escrever ("Erro: $R1$ deve ser $\neq 0$ ") Senão:
 $Rx \leftarrow (R3 \times R2) / R1$
 $R_{\text{total}} \leftarrow ((R1 + Rx) \times (R2 + R3)) / (R1 + Rx + R2 + R3)$
 $I_{\text{total}} \leftarrow V_{\text{fonte}} / R_{\text{total}}$
 $P_{\text{total}} \leftarrow V_{\text{fonte}} \times I_{\text{total}}$
- **Condicional de Equilíbrio:** Se ($R1 \times Rx = R2 \times R3$) então "Ponte equilibrada" senão "Ponte desequilibrada".

CÓDIGO PYTHON

```
R1 = float(input("Digite R1 (Ω): "))
R2 = float(input("Digite R2 (Ω): "))
R3 = float(input("Digite R3 (Ω): "))
Vfonte = float(input("Digite a Tensão da Fonte (V): "))

if R1 == 0:
    print("Erro: R1 tem que ser ≠ 0")
else:
    Rx = (R3 * R2) / R1
    R_total = ((R1 + Rx) * (R2 + R3)) / (R1 + Rx + R2 + R3)
    I_total = Vfonte / R_total
    P_total = Vfonte * I_total

    print(f"
    Resistência Rx: {Rx:.2f} Ω"
    print(f"Resistência total: {R_total:.2f} Ω")
```

```

print(f"Corrente total: {I_total:.5f} A")
print(f"Potência total: {P_total:.4f} W")

if (R1 * Rx) == (R2 * R3):
    print("A ponte está equilibrada")
else:
    print("A ponte está desequilibrada")

```

Exercício 3: Cálculo Tributário em Cascata

Avaliação Crítica: O algoritmo calcula os impostos em formato cascata (onde o imposto anterior integra a base de cálculo do próximo). A estrutura lógica está correta. No código Python, adicionou-se a fórmula matemática explícita para obter a **carga_{total}** percentual solicitada na saída.

PSEUDOCÓDIGO TRANSCRITO

- **Variáveis:** *preco_fabrica*, *aliq_ipi*, *aliq_icms*, *aliq_pis_cofins* (Reais)
- **Processamento:**
 - $ipi \leftarrow preco_fabrica \times aliq_ipi / 100$
 - $base_icms \leftarrow preco_fabrica + ipi$
 - $icms \leftarrow base_icms \times aliq_icms / 100$
 - $base_pc \leftarrow base_icms + icms$
 - $pis_cofins \leftarrow base_pc \times aliq_pis_cofins / 100$
 - $preco_final \leftarrow base_pc + pis_cofins$

CÓDIGO PYTHON

```

preco_fabrica = float(input("Digite o preço de fábrica (R$): "))
aliq_ipi = float(input("Digite a alíquota do IPI (%): "))
aliq_icms = float(input("Digite a alíquota do ICMS (%): "))
aliq_pis_cofins = float(input("Digite a alíquota do PIS/COFINS (%): "))

ipi = preco_fabrica * (aliq_ipi / 100)
base_icms = preco_fabrica + ipi
icms = base_icms * (aliq_icms / 100)
base_pc = base_icms + icms
pis_cofins = base_pc * (aliq_pis_cofins / 100)
preco_final = base_pc + pis_cofins

carga_total = ((preco_final - preco_fabrica) / preco_final) * 100

print(f"
Preço Final: R$ {preco_final:.2f}")

```

```
print(f"Carga Tributária Total: {carga_total:.2f}%")
```

3

Exercício 4: Laudo de Sondagem de Solo (SPT)

Avaliação Crítica: Uso adequado de operadores lógicos (**OU / E**) para garantir a segurança da estrutura. Para evitar redundâncias de múltiplas saídas conflitantes na execução real, o código Python foi otimizado usando uma estrutura encadeada (**if-elif-else**) mutuamente exclusiva.

PSEUDOCÓDIGO TRANSCRITO

- **Variáveis:** *spt* (Inteiro), *cap_carga* (Real)
- **Regras de Decisão:**
 - Se (*spt* < 5) OU (*cap_carga* < 80) → Inapto (Reforço necessário)
 - Se (*spt* < 10) OU (*cap_carga* < 150) → Restrito (Fundação especial)
 - Se (*spt* ≥ 30) E (*cap_carga* ≥ 300) → Excelente
 - Senão → Adequado (Fundação convencional)

CÓDIGO PYTHON

```
spt = int(input("Digite o valor de SPT: "))
cap_carga = float(input("Digite a capacidade de carga (kPa): "))

if spt < 5 or cap_carga < 80:
    print("Laudo: Inapto - solo muito fraco - reforço necessário")
elif spt < 10 or cap_carga < 150:
    print("Laudo: Restrito - fundação especial recomendada")
    if spt < 10: print(f"Parâmetro Limitante: SPT baixo ({spt})")
    if cap_carga < 150: print(f"Parâmetro Limitante: Carga baixa ({cap_carga} kPa)")
elif spt >= 30 and cap_carga >= 300:
    print("Laudo: Excelente - solo de alta capacidade")
else:
    print("Laudo: Adequado - fundação convencional viável")
```

Exercício 5: Controle de Qualidade e Esteira

Avaliação Crítica: Excelente divisão em dois blocos de teste industriais independentes. A lógica do "Desafio Extra" funciona perfeitamente como um refinamento restritivo integrado à checagem de desvio da Parte A.

PSEUDOCÓDIGO TRANSCRITO

- **Parte A (Solda):** Verifica se o desvio está dentro de $\pm 8\%$ da resistência nominal.
- **Parte B (Esteira):** Classifica a velocidade atual em relação à velocidade alvo (tolerância de

$\pm 10\%$). CÓDIGO PYTHON

```
# Parte A - Solda
res_nominal = float(input("Resistência Nominal: "))
res_medida = float(input("Resistência Medida: "))
tolerancia_abs = res_nominal * 0.08
desvio = res_medida - res_nominal

if -tolerancia_abs <= desvio <= tolerancia_abs:
    print("Resultado: Solda Aprovada")
else:
    pct_desvio = (desvio / res_nominal) * 100
    print(f"Reprovada. Desvio: {desvio:.2f} MPa ({pct_desvio:.2f}%)")

# Parte B - Esteira
v_atual = float(input("Velocidade Atual: "))
v_alvo = float(input("Velocidade Alvo: "))
```

```
if v_atual > (v_alvo * 1.10):  
    print("Velocidade Alta: Risco de posicionamento incorreto")  
elif v_atual < (v_alvo * 0.90):  
    print("Velocidade Baixa: Gargalo na produção")  
else:  
    print("Velocidade OK")
```